

CCF2026开源创新大赛

Mooncake 赛题2初赛技术报告

Mooncake Store 碎片感知分配优化

Fragmentation-Aware Allocation for Mooncake Store

参赛方向	赛题2：优化Mooncake Store吞吐性能、高可用功能和可扩展性，优化SGLang HiCache+Mooncake Store 性能
作品名称	Mooncake Store 碎片感知分配策略
队伍名称	KVCache Forge
队伍成员	罗荣裕（队长）、刘神舟、王英颖、杨俊
GitHub 仓库	https://github.com/Lorry1024/ccf2026-mooncake-fragmentation-aware
GitLink 仓库	https://gitlink.org.cn/lorry22/ccf2026-mooncake-fragmentation-aware
上游PR	https://github.com/kvcache-ai/Mooncake/pull/2797
PR头提交	0123fa1 Fix fragmentation-aware allocation test setup
报告日期	2026年7月10日

本报告与仓库中的patch、复现实验、验证日志和展示PPT共同构成初赛提交材料。

摘要

Mooncake Store 是 Mooncake 体系中支撑 KVCache 对象存储和复用的关键组件。现有 `free_ratio_first` 策略根据 `segment` 总空闲比例进行排序，能够改善整体空间利用均衡，但在混合大小 KVCache 对象和长时间运行的场景下，总空闲空间可能被切分为多个小空洞，无法真实反映当前大对象是否可以被连续放入。本文提出并实现 `fragmentation_aware` 策略，在保持默认行为不变和有界候选采样不变的前提下，将最大连续空闲区域、连续性比例和当前请求可容纳性纳入排序逻辑，使 Master 优先选择能够直接满足请求的 `segment`。

本作品已形成 Mooncake 上游 draft PR，当前 PR 头提交 `0123fa1` 已通过 GitHub Actions，结果为 26 个检查成功、1 个检查跳过。确定性专题仿真显示，在 6 个大对象请求上，`free_ratio_first` 的首选 `segment` 可直接容纳次数为 0/6，而 `fragmentation_aware` 为 6/6；fallback 尝试次数从 11 降至 0；平均候选 `segment` 数量保持 5.00；本地单次决策耗时从 121.00ns 增加到 138.93ns，额外开销约 17.93ns。该结果说明，新策略能够以很小的决策成本提升碎片化场景下的分配稳定性。

关键词： Mooncake Store；KVCache；碎片感知；连续空闲区域；分配策略；SGLang HiCache

模块完成情况

表 1 初赛阶段模块完成情况

模块	完成情况
问题分析	明确 free_ratio_first 在碎片化大对象分配中的误判模式，形成最小复现和专题仿真。
策略设计	设计 fragmentation_aware 排序规则，优先考虑 can_fit、连续性评分和最大连续空闲区域。
代码实现	在 Mooncake Store 中新增策略类型、策略类、配置解析、启动参数和 benchmark 矩阵。
测试验证	新增碎片化单元测试，保留 preferred segment 语义，PR 通过上游 GitHub Actions。
材料整理	完成中文 Markdown 材料、LaTeX 技术报告、展示 PPT、复现实验源码和提交包。
展示视频	5 分钟作品展示视频已录制并上传，链接与提取码见 README.md 和 SUBMISSION.md。
后续工作	真实 RDMA 和 SGLang HiCache 端到端 benchmark。

目录

1	赛题理解与项目定位	1
1.1	本文贡献	1
1.2	技术路线总览	2
2	原有机制与问题分析	2
2.1	Mooncake Store 分配链路 with 关键抽象	2
2.2	KVCache 对象生命周期与碎片来源	3
2.3	现有策略的能力边界	4
2.4	总空闲比例误判的形成机制	4
2.5	误判带来的性能影响	4
2.6	为什么不能只依赖 fallback	5
2.7	本项目抽象出的优化需求	5
3	碎片感知分配策略设计	6
3.1	设计目标与非目标	6
3.2	策略输入、输出与不变量	6
3.3	问题形式化	7
3.4	候选元数据与评分字段	7
3.5	排序规则	8
3.6	权重选择的工程解释	9
3.7	候选排序示例	9
3.8	完整策略流程	10
3.9	多副本场景下的处理	11
3.10	退化行为与边界保护	11
3.11	正确性与不变量论证	11
3.12	复杂度与并发安全分析	12
4	实现路径	12
4.1	代码改动总览	12
4.2	策略类接入	14
4.3	核心执行流程	14

4.4	配置和工厂接入	16
4.5	兼容性、灰度与回滚	17
4.6	单元测试设计	17
4.7	测试修复与CI闭环	18
4.8	文档、benchmark与复现材料	18
5	验证与效果	18
5.1	验证思路	18
5.2	CI验证	19
5.3	专题仿真指标	21
5.4	结果解释	22
5.5	证据有效性与威胁分析	22
6	与赛题要求的对应	23
6.1	评审证据链	24
7	使用与复现	25
7.1	启用策略	25
7.2	应用补丁	25
7.3	回滚补丁	26
7.4	复现实验	26
8	边界与后续工作	27
9	结论	27
10	提交材料清单	27
11	参考链接	28

1 赛题理解与项目定位

Mooncake 赛题2要求围绕Mooncake Store吞吐性能、高可用功能、可扩展性以及SGLang HiCache+Mooncake Store性能开展优化。该要求并不只局限于网络传输或外层推理框架，也包含Store后端内部影响吞吐稳定性和资源利用率的关键路径。对KV-Cache场景而言，Store中对象的分配位置会影响一次写入是否能顺利落地，也会影响后续fallback、重试、淘汰和副本放置压力。

本项目选择的切入点是Mooncake Store分配策略层。该方向具有三个特点：第一，它位于真实Mooncake Store代码路径中，能够通过PR形式提交给上游；第二，它对SGLang HiCache等前端具有后端相关性，因为这些前端将KVCache对象交给Store时需要依赖Store的放置决策；第三，它可以在不重写协议、不引入重依赖、不改变默认行为的条件下完成可验证优化。

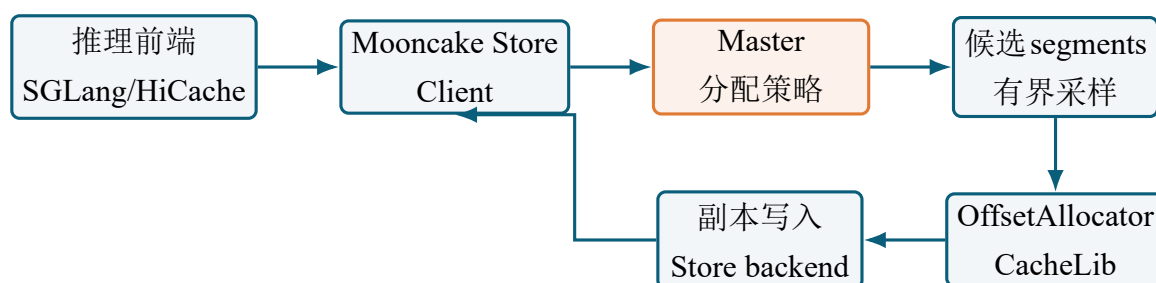


图 1 本项目在Mooncake Store路径中的位置

1.1 本文贡献

本文围绕Mooncake Store在长期运行和混合尺寸KVCache写入下的碎片化问题，完成了从问题识别、算法设计、上游实现到可复现验证的闭环工作。第一，本文将“总空闲容量充足但当前对象仍无法落地”的现象明确归因于空闲空间形态与请求尺寸之间的不匹配，并指出仅以总空闲比例进行候选排序时，无法区分“空闲空间集中且可用”和“空闲空间分散且不可用”两种状态。该分析把抽象的碎片化问题落实为Master分配路径中的一次可观察误判，为策略设计提供了明确对象。

第二，本文提出并实现可选的**fragmentation_aware**策略。该策略在现有有界候选集合内读取总空闲空间、最大连续空闲区域等元数据，先以当前请求是否可被容纳作为硬排序条件，再以连续性比例和总空闲比例的加权评分区分同类候选。策略不改变底层allocator的最终判定权，不扩大全局搜索范围，也不替代原有fallback，而是通过改进尝试顺序减少明显无效的首选分配。

第三，本文完成了面向上游Mooncake工程体系的完整接入，包括策略类型、配置解析、工厂创建、命令行帮助、设计文档、部署说明、单元测试和benchmark矩阵。实

现继续遵守 `preferred`、`excluded`、多副本去重和 `best-effort fallback` 语义，默认策略保持不变，从而使优化能够以显式开关进行灰度验证和快速回滚。

第四，本文建立了分层证据链：使用定向单元测试验证碎片反例和兼容语义，使用上游 GitHub Actions 验证构建、格式、文档及回归测试，使用确定性专题仿真比较首选成功、`fallback` 次数、候选规模和决策开销。报告同时明确区分策略层证据与真实 RDMA 集群、SGLang HiCache 端到端压测结论，保证成果的有效范围清晰、复现路径完整。

1.2 技术路线总览

本项目的技术路线可以概括为“先把碎片化问题显式建模，再把模型压缩为可在现有分配路径中低成本执行的排序信号”。我们没有从修改网络协议、重写存储后端或引入全局整理线程入手，原因是这些方向虽然可能带来更大范围的能力提升，但会显著增加验证复杂度，并且容易影响 Mooncake Store 已有部署行为。赛题初赛更需要一个边界清晰、证据充分、能够被上游项目评审的优化点。因此，本项目把关注点收敛到 Master 侧 `segment` 选择过程：当 Store 面对一个已知大小的 KVCache 对象时，应该优先选择“当前真的能放下”的 `segment`，而不是只选择“总空闲比例看起来更高”的 `segment`。

从工程路径看，完整工作被拆成六步。第一步，阅读 Mooncake Store 已有分配策略，确认 `random`、`free_ratio_first` 等策略的职责边界。第二步，构造最小碎片化反例，证明总空闲比例和连续可用空间之间存在可观察的不一致。第三步，设计 `fragmentation_aware` 评分字段，把最大连续空闲区域、连续性比例和当前请求大小引入候选排序。第四步，把策略接入 Mooncake Store 正式配置路径，而不是停留在离线脚本或实验代码中。第五步，补充单元测试、`benchmark` 矩阵和文档说明，保证策略可使用、可回滚、可被评审。第六步，通过上游 PR 和 GitHub Actions 完成开源项目层面的基本验证。该路线对应赛题 2 的三个关键约束：优化点位于 Mooncake Store 主路径，策略开销受有界候选采样约束控制，并且默认行为、`preferred/excluded` 语义和 `fallback` 路径均保持不变。

2 原有机制与问题分析

2.1 Mooncake Store 分配链路与关键抽象

Mooncake Store 在写入 KVCache 对象时，需要为对象的每个副本选择可写入的 `segment`。上层调用并不直接关心底层空闲块的形态，而是把对象大小、副本数、`preferred segment` 和 `excluded segment` 等约束交给 Master 侧分配逻辑。Master 再调用具体分配策

略，从可用 segment 集合中选出候选，并通过对应 allocator 完成空间申请。这个过程看似只是“选择一个 segment”，但实际承担了三类语义：

1. **容量语义**。当前 segment 是否拥有足够空间，决定对象是否可以一次性落地。
2. **副本语义**。多副本写入需要避免重复使用同一 segment，并在候选不足时保留 best-effort fallback。
3. **调度语义**。preferred segment 表示上层希望优先使用的放置位置，excluded segment 表示本次请求不能选择的位置。

因此，分配策略不是孤立的性能小优化，而是 Store 吞吐稳定性、资源利用率和高可用副本放置之间的连接点。本项目的接入点在 Master 侧策略工厂和配置解析路径；被排序的对象是 segment；真正判断空间能否占用的是 segment 内部 allocator；preferred segments 和 excluded segments 则分别代表上层显式优先级和本次请求的禁止集合。如果策略选中了“总空闲很多但无法容纳当前对象”的 segment，后续 allocator 仍会拒绝分配，系统只能继续尝试其他 segment 或走 fallback 路径。该失败不会直接制造数据错误，但会增加一次无效尝试，并可能放大为锁竞争、重试、尾延迟和副本放置压力。

2.2 KVCache 对象生命周期与碎片来源

KVCache 对象的存储行为和普通定长块写入不同。推理系统中不同请求的上下文长度、batch 形态、prefix 共享程度和缓存淘汰时机都可能不同，导致写入 Store 的对象大小呈现明显差异。一个较短请求可能只产生较小的 KVCache 对象，一个长上下文请求可能产生更大的对象；多轮对话或 prefix 复用又会让对象生命周期不完全同步。当这些对象交错写入、交错释放时，同一个 segment 内部就会逐渐形成不规则的空闲区域。

可以把 segment 想象成一个连续地址空间。初始状态下，segment 中空闲区域通常比较完整，任何大小合理的对象都容易落入。运行一段时间后，不同大小对象被插入其中；再运行一段时间后，部分对象被释放，释放位置不一定相邻。此时虽然空闲字节总量可能很多，但这些字节被分散在多个小区间中。碎片来源主要包括对象大小混合、生命周期不同步、多副本写入、前缀复用命中后的非同步淘汰以及长时间运行中的持续分配释放。对 allocator 而言，当前请求是否可分配，取决于是否存在一个足够大的连续空闲区间，而不是所有小区间相加是否足够。

这一点正是赛题2中“Store 吞吐性能”和“SGLang HiCache+Mooncake Store 性能”之间的连接处。HiCache 等前端把 KVCache 对象交给 Store，并期望后端尽快完成存放和复用。如果 Store 在碎片化场景下反复优先选择不可容纳的大对象位置，就会增加后端分配路径的不确定性。即使最终能够通过 fallback 找到可用位置，前面失败的尝试仍然消耗了决策、锁、状态检查和日志路径资源。

2.3 现有策略的能力边界

Mooncake Store 已有多种分配策略。默认 `random` 策略开销低、行为简单，适合通用场景；`free_ratio_first` 策略按照总空闲比例排序，目标是让更空闲的 `segment` 优先吸收写入，从而改善整体空间利用均衡；`ssd_free_ratio_first`、`cx1` 和 `local_first` 等策略则面向特定介质或拓扑。就赛题2而言，`free_ratio_first` 是最重要的对比基线，因为它已经使用了空间信息，却仍然没有识别 `segment` 内部碎片。

`free_ratio_first` 的核心假设可以概括为：总空闲比例越高，当前请求越可能分配成功。该假设在对象大小比较均匀、释放行为比较规则、`segment` 内部空闲区较少被切碎时通常成立。但 KVCache Store 的运行状态并不总是满足这些条件。长上下文推理、不同 `batch` 形态、前缀复用和缓存淘汰会让对象大小差异扩大；对象生命周期不同步又会让释放位置分散。最终，一个 `segment` 可能拥有较高总空闲字节数，但这些空间被拆成多个小洞，无法提供一个足够大的连续区域。

2.4 总空闲比例误判的形成机制

在 `allocator` 需要连续空间的前提下，“总空闲空间”和“最大连续空闲区域”是两个不同指标。前者衡量还有多少空间未被使用，后者衡量当前请求能否被一次性放入。碎片化场景下，两者可能明显背离。图2展示了典型情况：上方 `segment` 的绿色空闲块数量多、总量不低，但每个块都较小；下方 `segment` 的总空闲量略低，却拥有足够大的连续区域。

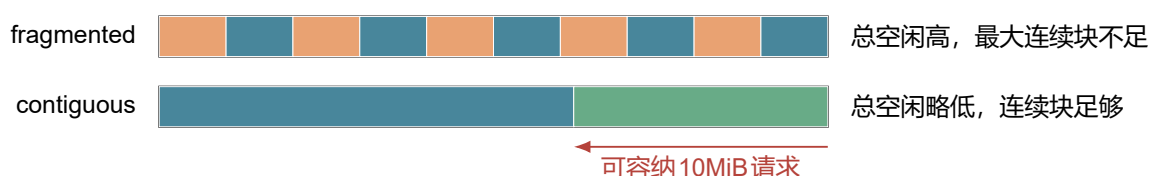


图 2 总空闲空间与最大连续空闲区域可能不一致

最小复现数据如表2所示。`free_ratio_first` 会优先选择总空闲空间为 16MiB 的 `fragmented`，因为它的总空闲比例更高；但这个 `segment` 的最大连续空闲区域只有 8MiB，无法容纳 10MiB 请求。相反，`contiguous` 虽然总空闲空间只有 12MiB，却可以直接完成本次分配。

2.5 误判带来的性能影响

该问题的关键不在于最终一定失败，而在于首选候选质量下降。典型链路是：排序阶段把高总空闲但碎片化严重的 `segment` 放在前面，`allocator` 阶段发现最大连续区域

表 2 最小碎片化复现场景

segment	总容量	总空闲	最大连续空闲	10MiB 请求
fragmented	32MiB	16MiB	8MiB	不能容纳
contiguous	32MiB	12MiB	12MiB	可以容纳

小于请求大小，当前副本只能继续尝试其他 segment；如果多个副本同时处于碎片化状态，失败尝试会被放大。对推理系统而言，这类失败尝试会影响 Store 写入路径的稳定性，进而影响 KVCache 对象的后端落盘、复用和淘汰压力。

2.6 为什么不能只依赖 fallback

一种直观疑问是：既然 `free_ratio_first` 选错后仍然可以继续尝试其他 segment，为什么还需要新增策略？原因在于 fallback 解决的是“最终能否找到位置”，而不是“首选路径是否稳定”。在高吞吐推理服务中，Store 面对的是连续不断的对象写入请求。一次请求多一次失败尝试看似开销有限，但在高并发、多副本、大对象和长时间运行组合下，会形成累计影响。

首先，fallback 通常意味着前一次候选已经做过元数据读取、排除集合判断、allocator 尝试和失败处理。其次，多副本请求中每个副本都要避免重复使用同一 segment，因此一个副本的失败会改变后续副本的候选空间。再次，fallback 路径本质上是补救逻辑，它可以提高可用性，但不应成为碎片化场景下的常态路径。更理想的做法是在排序阶段就把明显不可容纳的候选排到后面，让系统优先尝试成功概率更高的 segment。也就是说，fallback 解决“失败后如何补救”，而碎片感知排序解决“第一次尝试能否更接近正确位置”。

因此，本项目并不是替代 fallback，而是减少不必要 fallback。保留 fallback 是为了处理并发状态变化、候选不足、元数据估计不精确等不可避免情况；新增 `fragmentation_aware` 是为了让正常路径更少走到补救路径。这种关系更符合生产系统设计：主路径尽量准确，兜底路径必须存在，但不应成为常态。

2.7 本项目抽象出的优化需求

基于上述分析，本项目没有选择重写 Store 协议或引入复杂全局整理机制，而是把问题收敛为“在现有候选集合内提高排序质量”。这个抽象有明确边界：它不尝试移动已有对象，不做在线压缩，不改变副本协议，也不扩大候选扫描范围；它只利用 allocator 已经能提供或可近似得到的元数据，让策略在面对大对象请求时先识别“当前是否真的能放下”。因此，优化需求可以拆成四点：

1. 在排序中显式加入最大连续空闲区域，而不是只看总空闲比例。
2. 在当前请求大小已知的情况下，优先选择`can_fit=true`的候选。
3. 保持原有采样规模和 fallback 语义，避免策略本身成为新的扩展性瓶颈。
4. 保持配置可选和默认行为不变，便于灰度、回滚和上游接受。

3 碎片感知分配策略设计

3.1 设计目标与非目标

`fragmentation_aware`的目标是提高碎片化场景下的首选 `segment` 可用性。具体来说，它希望在混合大小对象场景中优先选择最大连续空闲区域能够容纳当前请求的 `segment`，沿用 `free_ratio_first` 的有界候选思想，只改变候选排序信号，不扩大搜索范围；同时保留 `preferred segment`、`excluded segment`、多副本去重和 `best-effort fallback` 语义，并作为显式可选策略接入，不改变 `random` 默认行为。它的非目标也很明确：不进行对象迁移、在线压缩或全局碎片整理，不修改 Mooncake Store 客户端协议、`metadata` 协议或后端存储格式，也不把初赛阶段的策略层验证夸大为真实 RDMA 集群或 SGLang HiCache 端到端压测结论。设计时刻意区分目标和非目标，是为了避免优化范围失控。

3.2 策略输入、输出与不变量

`fragmentation_aware` 并不是独立调度器，而是 Mooncake Store 分配策略体系中的一个可选实现。因此，它的输入和输出必须严格服从已有接口约束。策略输入主要包括当前请求的 `slice` 大小、副本数量、`allocator manager` 中的 `segment` 视图、`preferred segment` 列表和 `excluded segment` 集合。策略输出不是全局放置计划，而是一组已成功申请的 `buffer` 句柄；每个成功结果都对应一个实际 `allocator` 申请。换言之，评分阶段只决定尝试顺序，真正的空间占用仍以 `allocator` 返回结果为准。

本项目在设计中保留三个不变量。第一，任何被 `excluded` 的 `segment` 都不能被选中。第二，同一次多副本分配中，已经成功使用的 `segment` 会进入 `used` 集合，后续副本不能重复选择。第三，`preferred segment` 具有显式优先级，只要 `preferred` 可用并能完成分配，就不进入后续评分排序。这些不变量保证新策略不会改变调用方已经依赖的放置语义。请求大小 `slice_length` 只用于判断候选是否具备足够连续空间，副本数 `replica_num` 只决定最终需要返回多少个 `buffer`；策略内部不会把评分结果当作空间占用事实。

3.3 问题形式化

为了说明新策略究竟优化什么，设当前请求需要一段大小为 $r > 0$ 的连续空间，采样得到的 **segment** 候选集合为 $\mathcal{C}$ 。对于任意候选 **segment** s ，记其 **allocator** 可见的空闲区间集合为 $\mathcal{F}_s = \{(b_i, e_i)\}$ ，其中 b_i 和 e_i 分别表示第 i 个空闲区间的起止位置。由此可以定义总空闲空间、最大连续空闲区域和当前请求可容纳性：

$$total_free(s) = \sum_{(b_i, e_i) \in \mathcal{F}_s} (e_i - b_i) \quad (1)$$

$$largest_free_region(s) = \max_{(b_i, e_i) \in \mathcal{F}_s} (e_i - b_i) \quad (2)$$

$$can_fit(s, r) = \begin{cases} 1, & largest_free_region(s) \geq r, \\ 0, & largest_free_region(s) < r. \end{cases} \quad (3)$$

总空闲空间只回答“还剩多少字节”，最大连续空闲区域则回答“是否存在一个足够大的完整位置”。当 $total_free(s) \geq r$ 但 $largest_free_region(s) < r$ 时，候选在容量总量意义上看似充足，却无法满本次连续分配。这正是 **free_ratio_first** 可能误判的状态。因此，本项目的首要优化目标不是单纯最大化 $total_free$ ，而是在有效候选集合 $\mathcal{C}' = \mathcal{C} \setminus (\mathcal{E} \cup \mathcal{U})$ 中，优先最大化 $can_fit(s, r)$ 。其中 $\mathcal{E}$ 为 **excluded** 集合， $\mathcal{U}$ 为本次多副本分配已经使用的 **segment** 集合；**preferred** 集合仍在进入 $\mathcal{C}'$ 排序前按原有语义单独尝试。

在多个候选具有相同可容纳状态时，策略才进一步比较连续性比例、总空闲比例、最大连续空闲区域和稳定序号。换言之，整体选择是一个分层的字典序优化，而不是把所有目标压缩为一个不可解释的黑盒分数：可容纳性保证当前请求的直接可行性，综合评分兼顾碎片形态与空间均衡，稳定 **tie-break** 保证相同状态下结果可复现。最终 **allocator** 申请仍是成功与否的唯一依据，形式化判定只负责提高候选尝试顺序的质量。

3.4 候选元数据与评分字段

策略对每个采样候选 **segment** 计算五类字段。这里的关键是把“容量总量”和“空间形态”拆开：**free_ratio** 继续描述整体空闲程度，**largest_free_region** 和 **contiguity_ratio** 描述空闲区域是否集中，**can_fit** 则把当前请求大小纳入判断。

在实现中，若某类 **allocator** 无法提供精确的最大连续空闲区域，策略退化为使用可用空闲空间作为估计值。这样做的原因是兼容性优先：新策略不能要求所有 **allocator** 一次性改造完成。对能提供精确碎片信息的 **OffsetAllocator**，策略会使用真实 **getLargestFreeRegion()**；对无法提供精确值的场景，行为退化为接近总空闲视角，不会阻断已有部署。

表 3 候选segment评分字段

字段	计算方式	设计含义
total_free	所有allocator可用空间之和	描述segment剩余空间总量
largest_free_region	allocator报告的最大连续空闲区域最大值	描述当前对象是否存在可落入的连续块
free_ratio	$\text{total_free} / \text{total_capacity}$	保留原有空间均衡信号
contiguity_ratio	$\text{largest_free_region} / \text{total_free}$	衡量空闲空间是否集中
can_fit	$\text{largest_free_region} \geq \text{request_size}$	直接判断当前请求是否可被候选容纳

3.5 排序规则

评分公式如下：

$$\text{free_ratio} = \frac{\text{total_free}}{\text{total_capacity}} \quad (4)$$

$$\text{contiguity_ratio} = \frac{\text{largest_free_region}}{\text{total_free}} \quad (5)$$

$$\text{score} = 0.70 \times \text{contiguity_ratio} + 0.30 \times \text{free_ratio} \quad (6)$$

其中0.70和0.30的权重体现了本策略的取舍：在碎片化优化场景下，空闲空间是否集中比总空闲比例更关键；但完全忽略总空闲比例也不合理，因为当多个候选都能容纳当前请求时，更高总空闲比例有助于维持segment之间的负载均衡。因此排序采用分层规则，而不是只用单一score：

1. **can_fit=true**的候选优先于**can_fit=false**的候选。
2. 同为可容纳或同为不可容纳时，**score**高者优先。
3. **score**相近时，**largest_free_region**大者优先。
4. 仍无法区分时，**free_ratio**高者优先。
5. 最后使用稳定tie-break，避免同一输入下排序非确定。

Listing 1 fragmentation_aware候选排序伪代码

```

for candidate in sampled_segments:
    total_free = sum_free_bytes(candidate)
    total_capacity = sum_capacity(candidate)

```

```
largest = largest_contiguous_free_region(candidate)
free_ratio = total_free / total_capacity
contiguity = largest / total_free
can_fit = largest >= request_size
score = 0.70 * contiguity + 0.30 * free_ratio

sort candidates by:
    can_fit desc,
    score desc,
    largest desc,
    free_ratio desc,
    stable_index asc
```

3.6 权重选择的工程解释

评分公式中 `contiguity_ratio` 权重为 0.70，`free_ratio` 权重为 0.30。这个选择不是为了追求复杂模型，而是为了表达一个工程优先级：在碎片化问题上，连续空间形态应当成为主要信号，但总空闲比例仍然不能被完全丢弃。如果只按最大连续空闲区域排序，策略可能长期偏向少数拥有大连续块的 `segment`，导致整体空间负载不均衡；如果只按总空闲比例排序，则会回到 `free_ratio_first` 的误判问题。0.70 和 0.30 的组合让策略在两者之间形成明确偏好：先保证当前请求可放入，再兼顾空间均衡。

更重要的是，`fragmentation_aware` 并不直接用 `score` 替代所有排序条件。它先比较 `can_fit`，再比较 `score`。这意味着当候选 A 无法容纳当前请求、候选 B 可以容纳当前请求时，即使 A 的总空闲比例更高，也不会排在 B 前面。这个设计避免了“高分但不可用”的情况。`score` 只在同一 `fit` 状态内部发挥作用，用于区分多个都可用或多个都不可用的候选。相比只看总空闲比例，它能判断当前请求是否存在足够连续空间；相比只看最大连续空闲区域，它仍保留空间均衡信号；相比只看连续性比例，它不会忽略绝对空闲容量。代价是多一次碎片元数据读取和排序比较，但该成本受候选规模约束。

3.7 候选排序示例

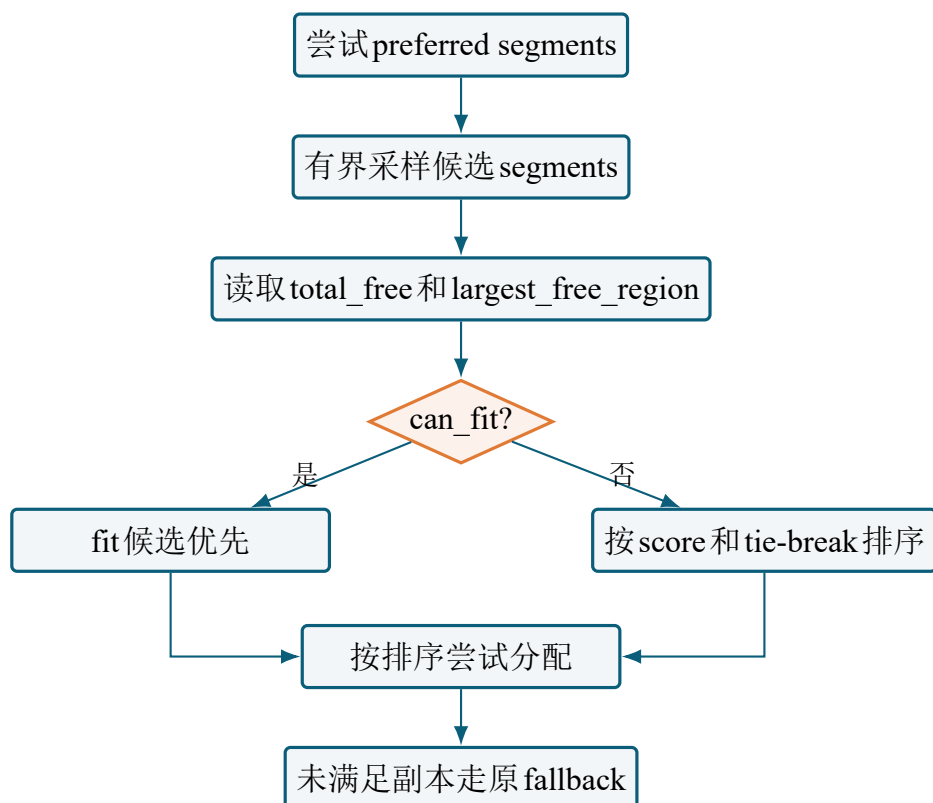
为了更直观说明策略行为，可以考虑三个候选 `segment`。Segment A 总空闲最高，但最大连续块不足；Segment B 总空闲中等，最大连续块刚好满足请求；Segment C 最大连续块更大，但总空闲比例较低。若当前请求为 10MiB，则 A 虽然总空闲高，但 `can_fit=false`，会排在 B 和 C 之后；B 和 C 都能容纳请求，再根据综合 `score` 和最大连续区域排序。这样排序结果反映的是“先避免明显失败，再在可用候选中做质量选择”。

表 4 候选排序示例

segment	总空闲	最大连续空闲	请求大小	can_fit	排序解释
A	20MiB	8MiB	10MiB	false	总空闲最高但不可容纳，排在可容纳候选之后
B	14MiB	10MiB	10MiB	true	可容纳且空闲较均衡，是优先候选之一
C	12MiB	12MiB	10MiB	true	可容纳且连续块更大，与B按score和tie-break比较

3.8 完整策略流程

图3展示了 `fragmentation_aware` 的完整流程。这里需要强调两点。第一，`preferred segment` 阶段不进入评分排序，而是沿用原有优先尝试语义；这是为了保证调用方显式指定的位置仍具有最高优先级。第二，评分排序只发生在采样候选上，且排序后仍逐个调用原有 `allocator` 尝试分配；如果候选不足以满足副本数，策略继续进入原有 `fallback` 路径。

图 3 `fragmentation_aware` 策略流程

3.9 多副本场景下的处理

Mooncake Store 并不只处理单副本写入。多副本场景下，策略需要同时满足“每个副本找到可用空间”和“同一次请求尽量不要重复使用同一 segment”两个条件。若只为第一个副本选择最优 segment，而忽略后续副本，可能导致后续副本候选空间不足。因此，`fragmentation_aware` 在每次成功分配后都会将该 segment 加入 `used` 集合。后续副本采样和分配时会跳过 `used segment`，从而保留已有策略中的副本去重语义。

这种处理方式也解释了为什么策略没有输出一一次性全局排序计划。Store 中的空间状态可能在并发请求之间快速变化，提前构造完整计划并不能保证全部成功。更稳妥的方式是：每个副本先读取 `excluded` 和 `used` 集合，优先尝试 `preferred segment`；随后对未排除、未使用的候选进行碎片感知排序；分配成功后立即把 segment 加入 `used` 集合；如果候选不足，则进入原有 `fallback` 路径。这样可以在不引入复杂全局锁和事务计划的情况下，保持较好的工程鲁棒性。

3.10 退化行为与边界保护

一个可合入上游的策略必须考虑不完整信息。并不是所有 `allocator` 都一定能提供精确的碎片信息；即使能够提供，评分后到实际分配之间也可能发生并发状态变化。因此，`fragmentation_aware` 从一开始就把碎片信息定位为“排序提示”，而不是“成功承诺”。当 `getLargestFreeRegion()` 不可用或返回未知值时，策略使用可用空闲空间进行保守估计；当排序后实际分配失败时，策略不会强行占用空间，而是继续尝试后续候选或 `fallback`。

这种设计有两个好处。第一，它避免新策略对所有 `allocator` 形成硬依赖，降低引入成本。第二，它把正确性仍然放在 `allocator` 最终检查上，避免因为元数据读取不精确导致空间一致性问题。换句话说，`fragmentation_aware` 优化的是“尝试顺序”，不是“容量判定权”。容量判定权仍然属于底层 `allocator`。

3.11 正确性与不变量论证

本节讨论的是策略层局部正确性，即在不改变 Mooncake Store 既有分配接口和 `allocator` 语义的前提下，新排序不会引入越权选择、重复副本或虚假分配成功；它不等价于对整个分布式存储系统进行形式化证明。论证基础是：评分结果只改变候选访问顺序，任何 `buffer` 都必须由原有 `allocateSingle` 路径实际申请成功后才能进入返回结果。

首先是约束保持。候选进入评分前已经应用 `excluded` 集合和 `used` 集合过滤，排序比较器不会重新引入被过滤对象，因此被调用方禁止的 `segment` 不会因为得分较高而被选择；每次副本分配成功后立即更新 `used` 集合，因此同一次请求的后续副本不会重

复使用该segment。preferred列表在普通候选评分之前按传入顺序尝试，只有无法完成所需分配时才进入碎片感知排序，所以调用方显式放置优先级保持不变。

其次是容量安全。can_fit来自某一时刻的元数据快照，可能因为统计精度或并发更新而失效，因此策略从不把can_fit=true解释为“已经预留成功”。排序完成后仍调用底层allocator执行带有既有同步和边界检查的真实申请。若候选状态在评分与申请之间发生变化，allocator可以拒绝本次申请，策略随后尝试下一候选或进入fallback。由此，陈旧元数据最多降低本次排序收益，不会导致越界占用或把失败伪装成成功。

再次是退化与活性保持。当allocator不能提供精确最大连续空闲区域时，策略退化为保守的近似信号；当所有采样候选都失败或候选数量不足时，控制流回到原有best-effort fallback。新策略没有删除旧有成功路径，也没有增加必须满足的新前置条件，因此在相同底层状态下不会因为缺少碎片元数据而永久阻塞请求。它可能无法在每种allocator上获得同等优化收益，但仍能保持原有可达的分配结果和失败语义。

最后是作用域有界。新策略只处理现有采样阶段提供的 k 个候选，不读取或修改跨请求全局状态，不引入后台整理线程，也不持有新的长期锁。其影响被限制在单次请求的候选排序阶段。上述约束保持、容量安全、退化活性和作用域有界四项共同构成了本实现可安全灰度和适合上游评审的主要依据。

3.12 复杂度与并发安全分析

fragmentation_aware没有引入新的全局数据结构，也没有在策略内部缓存跨请求状态。它只在每次分配时对采样候选读取allocator元数据，并对候选数组排序。若候选数量为 k ，则额外计算复杂度为 $O(k)$ ，排序复杂度为 $O(k \log k)$ 。由于 k 来自现有有界采样，而不是全量segment数 N ，策略开销不会随集群规模线性扫描所有segment。

并发方面，新策略不改变allocator的实际申请路径，也不绕过已有锁和状态检查。评分阶段读取的空闲信息只能作为排序依据，不能作为最终成功承诺；真正分配仍由allocator执行。因此，即使评分后其他线程改变了某个segment的空闲状态，最坏结果也只是该候选分配失败并继续尝试后续候选，不会破坏空间一致性。总体上，候选规模沿用有界采样以避免全局扫描，元数据读取只服务于排序，排序成本受候选数量控制；无精确碎片信息时退化为近似总空闲视角，不强制改造所有allocator。

4 实现路径

4.1 代码改动总览

当前PR补丁涉及9个文件，覆盖策略实现、类型系统、配置解析、启动参数、测试、benchmark、文档和CI辅助步骤。表5列出主要改动。

表 5 关键代码改动

文件	作用
mooncake-store/include allocation_strategy.h	新增 FragmentationAwareAllocationStrategy, 实现候选评分、排序、preferred 处理和 fallback 衔接。
mooncake-store/include types.h	新增 FRAGMENTATION_AWARE 枚举值, 让策略进入统一类型系统。
mooncake-store/include master_config.h	解析 allocation_strategy=fragmentation_aware, 非法值继续走原有报错路径。
mooncake-store/src master.cpp	更新启动参数帮助信息, 确保用户可以从命令行发现新策略。
mooncake-store/tests allocation_strategy_test.cpp	增加碎片化定向测试、preferred segment 测试和参数化覆盖。
mooncake-store benchmarks allocation_strategy_bench.cpp	将新策略加入分配策略 benchmark 矩阵, 便于后续真实环境对比。
docs/source/design mooncake-store.md	增加策略说明、适用场景和与其他策略的选择建议。
docs/source/deployment mooncake-store-deployment -guide.md	增加部署参数说明, 给出启用方式。
.github/workflows ci.yml	在 Go store binding 集成测试前释放 runner 磁盘空间, 避免无关环境失败。

图4给出本项目在 Mooncake 上游仓库中的公开 PR 页面。截图同时展示了 PR 编号 2797、draft 状态、目标分支 kvcache-ai:main、来源分支 Lorry1024:ccf-fragmentation-aware-allocation、4 次提交和 9 个改动文件，使代码来源、上游目标和评审入口能够被直接核验。

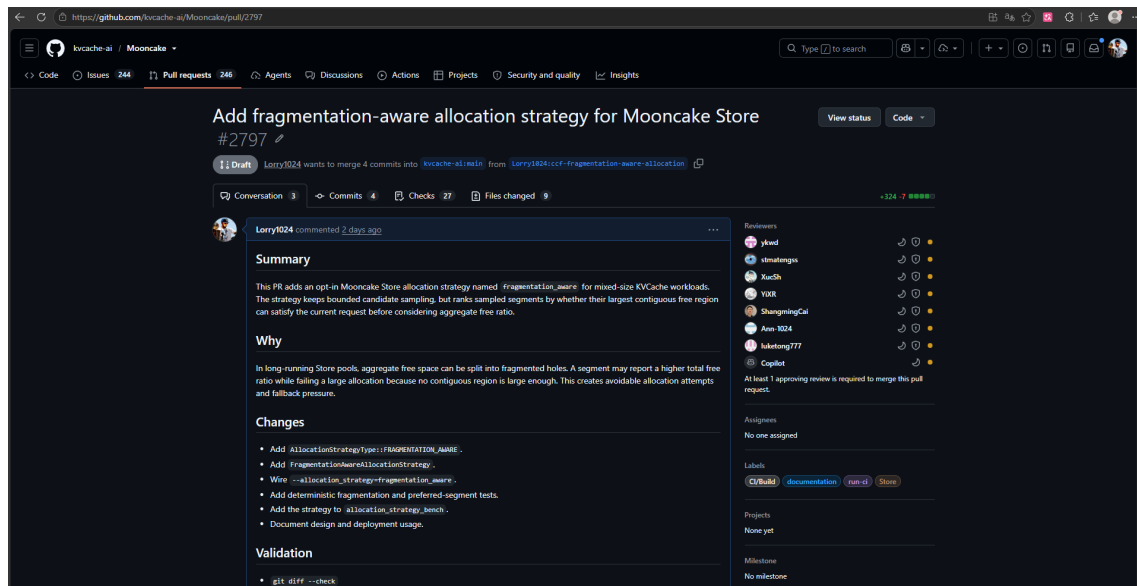


图 4 Mooncake 上游 PR#2797 总览

4.2 策略类接入

核心实现位于 `mooncake-store/include/allocation_strategy.h`。新类 `FragmentationAwareAllocationStrategy` 继承现有随机策略基础能力，而不是重写所有分配流程。这样做有两个好处：第一，可以复用原有单 segment 分配、随机 fallback 和排除集合处理；第二，可以把新增逻辑限制在候选评分和排序层，降低对主路径的侵入。

实现上，新策略先按原有语义处理 preferred segment。如果 preferred segment 没有被排除、没有被当前请求重复使用，并且 allocator 能够完成申请，则直接返回该分配结果。只有在 preferred 阶段无法满足全部副本时，策略才进入候选采样和碎片感知排序。排序后的候选按顺序尝试分配，成功后加入 used set，避免同一次多副本请求重复选择同一 segment。

4.3 核心执行流程

从代码路径看，`fragmentation_aware` 的核心执行可以分为五个阶段。第一阶段是预处理，把 excluded segment 和当前请求中已经成功使用的 segment 作为不可选集合。第二阶段是 preferred 尝试，按调用方传入顺序处理 preferred segment。第三阶段是候选

采样，沿用已有策略体系的有界采样思想，从剩余 segment 中得到候选集合。第四阶段是候选评分，对每个候选读取总容量、总空闲、最大连续空闲区域，并计算 `can_fit` 和 `score`。第五阶段是按排序结果尝试实际分配，成功则记录结果，失败则继续尝试后续候选；如果仍不能满足副本数，则回到原有 `fallback` 路径。

Listing 2 候选碎片元数据与请求相关评分的实际实现

```
const uint64_t free = capacity - used;
const uint64_t reported_largest =
    static_cast<uint64_t>(alloc->getLargestFreeRegion());
const uint64_t allocator_largest =
    reported_largest == kAllocatorUnknownFreeSpace
        ? free
        : std::min<uint64_t>(reported_largest, free);

total_capacity += capacity;
total_free += free;
largest_free_region =
    std::max(largest_free_region, allocator_largest);

const double free_ratio =
    static_cast<double>(total_free) /
    static_cast<double>(total_capacity);
const double contiguity_ratio =
    std::min(1.0,
        static_cast<double>(largest_free_region) /
        static_cast<double>(total_free));
const bool can_fit = largest_free_region >= slice_length;
const double score =
    0.70 * contiguity_ratio + 0.30 * free_ratio;
```

第一段代码是本项目相对 `free_ratio_first` 的关键差异。原策略主要使用 `free_ratio` 描述“还剩多少空间”，新策略进一步读取 `allocator` 报告的最大连续空闲区域，并通过 `can_fit` 把请求大小 `slice_length` 直接引入决策。若某类 `allocator` 暂时无法提供精确的连续空闲信息，代码使用当前总空闲量作为保守退化值；若能够提供，则用 `min(reported_largest, free)` 限制异常元数据，避免最大连续区超过实际总空闲量。连续性比例反映空闲空间是否集中，0.70 与 0.30 的权重使同为可容纳候选时优先保护连续空间，同时保留一定空间均衡能力。

Listing 3 候选排序优先级的实际实现

```
std::sort(candidates.begin(), candidates.end(),
    [](const Candidate& a, const Candidate& b) {
        if (a.can_fit != b.can_fit) {
```

```

        return a.can_fit > b.can_fit;
    }
    if (a.score != b.score) {
        return a.score > b.score;
    }
    if (a.largest_free_region !=
        b.largest_free_region) {
        return a.largest_free_region >
            b.largest_free_region;
    }
    if (a.free_ratio != b.free_ratio) {
        return a.free_ratio > b.free_ratio;
    }
    return a.name_idx < b.name_idx;
});

```

第二段代码给出完整排序优先级。`can_fit`是最高优先级，从根本上阻止“总空闲较高但当前对象无法落地”的候选排在可容纳候选之前；综合`score`只在相同可容纳状态内比较；最大连续区域与总空闲比例继续作为细粒度tie-break；最后使用`name_idx`形成确定性的稳定顺序，避免完全相同元数据下出现不可解释抖动。排序只改变尝试顺序，新策略没有接管底层空间管理，也没有绕过`allocateSingle`路径。`preferred`阶段、有界采样、`excluded/used`过滤、实际`allocator`申请和`best-effort fallback`均保留，因此评分元数据即使在并发下发生变化，也只会导致某次尝试失败并继续后续候选，不会突破空间安全边界。

4.4 配置和工厂接入

为了让策略成为正式可选项，而不是测试代码中的孤立类，PR在类型、配置和工厂三处同时接入：

1. 在`AllocationStrategyType`中新增`FRAGMENTATION_AWARE`枚举值。
2. 在`master_config.h`中识别字符串`fragmentation_aware`。
3. 在策略工厂中为`FRAGMENTATION_AWARE`返回新策略实例。
4. 在`master.cpp`和部署文档中更新`--allocation_strategy`可选值。

新策略通过Master启动参数显式启用：

```
mooncake_master --allocation_strategy=fragmentation_aware
```

由于默认策略保持为 `random`，现有部署不会因为合入该 PR 而改变行为。若线上灰度时发现新策略不适合某类负载，只需要切回 `random` 或 `free_ratio_first` 即可回滚。

4.5 兼容性、灰度与回滚

开源项目中的性能优化如果默认改变行为，评审风险会明显增大。Mooncake Store 面向真实推理服务，用户对默认策略的吞吐、稳定性和资源使用已经形成预期。因此，本项目把 `fragmentation_aware` 设计为显式启用策略，而不是替换默认策略。这样既可以让需要碎片优化的用户主动选择，也避免对现有部署产生隐式影响。

灰度使用时，可以先在测试环境或单个 Store 实例中启用新策略，观察失败分配次数、`fallback` 次数、写入延迟和空间利用率。如果指标改善，再扩大到更多实例；如果发现某类负载下新策略收益不足或元数据读取成本偏高，只需要把启动参数改回 `random` 或 `free_ratio_first` 即可回滚，不需要回滚二进制、修改客户端协议或迁移已有数据。若需要保留空间均衡对照，可继续使用 `--allocation_strategy=free_ratio_first` 作为基线。

4.6 单元测试设计

测试不是简单地检查“新策略能运行”，而是针对问题定义构造确定性场景。核心测试创建两个 segment: `fragmented` 和 `contiguous`。前者总空闲空间更高，但最大连续区域小于请求大小；后者总空闲空间较低，但最大连续区域可以容纳请求。断言目标是: `fragmentation_aware` 应该选择 `contiguous`，而不是被更高总空闲比例误导。

另一个关键测试是 `preferred segment` 语义。即使某个 `preferred segment` 在评分上不是最优，只要它没有被排除且可以满足请求，新策略仍应优先选择它。这一点对兼容性很重要，因为 `preferred segment` 通常来自上层调度或副本放置约束，不能被内部评分逻辑无条件覆盖。

碎片化测试的构造要避免一个常见陷阱：如果释放的是靠近尾部的块，`allocator` 可能把释放块和尾部剩余空间合并，导致最大连续空闲区域大于预期。这样测试看似构造了碎片，实际却没有形成“大量空闲但连续块不足”的状态。最终 PR 中的测试选择释放内部块，确保 `fragmented` 的总空闲空间较高，但最大连续空闲区域仍小于请求大小；同时构造 `contiguous`，使其总空闲空间较低但最大连续区域满足请求。这种测试更接近问题定义，能稳定验证排序信号是否正确。覆盖点包括碎片化选择、连续空间优先、`preferred` 保留、参数化基础分配语义和 `benchmark` 接入。

4.7 测试修复与CI闭环

PR创建后经历了三类CI问题并逐步修复。第一次失败来自代码格式检查，后续通过格式化修复。第二次失败来自GitHub runner磁盘不足，Go store binding集成测试在链接阶段出现空间不足；这不是策略逻辑失败，但会阻断PR验证，因此在对应job前增加了磁盘清理步骤。第三次失败来自最初碎片化测试构造不够严格：释放块与尾部空闲区域发生合并，导致最大连续区域大于预期，断言不稳定。后续测试改为构造5个8MiB块并释放内部块，避免尾部合并影响碎片形态。上述问题修复后重新触发CI，PR头提交0123fa1通过26个检查，1个检查跳过。

4.8 文档、benchmark与复现材料

除了代码本身，PR和比赛材料还补充了三类工程化内容。第一，在Mooncake官方文档中说明fragmentation_aware的适用场景和启用方式，避免评审只看到代码而不知道何时使用。第二，将新策略加入allocation_strategy_bench.cpp，让后续真实Store benchmark可以直接横向比较random、free_ratio_first和fragmentation_aware。第三，在比赛材料仓库中提供独立复现实验和日志，用轻量仿真固定问题模型，证明改进来自“排序信号变化”，而不是扩大候选规模或修改实验口径。

5 验证与效果

5.1 验证思路

本项目的验证分为三层。第一层是代码级验证，确认新策略能够编译、能够被配置解析、能够进入策略工厂，并且不会破坏已有分配策略测试。第二层是问题级验证，通过最小碎片化测试证明策略确实解决“总空闲比例高但最大连续空闲不足”的误判。第三层是专题仿真验证，通过多个请求和多个segment构造，观察首选成功次数、fallback尝试次数和单次决策成本。三层验证分别回答“能不能运行”“是否解决目标问题”“收益和成本如何”。

需要说明的是，初赛阶段的验证重点是策略层正确性和可复现性，而不是宣称已经完成完整生产压测。真实RDMA环境、真实SGLang HiCache端到端吞吐和长时间集群稳定性测试需要更多硬件和部署条件，适合作为决赛阶段继续推进。本报告中的数据口径均围绕策略层展开，不把离线仿真结果夸大为生产集群吞吐结论。代码级验证依赖单元测试、参数化测试和上游CI；问题级验证依赖碎片化定向测试；指标级验证依赖专题仿真和日志；工程级验证依赖文档、benchmark接入和PR流程。

5.2 CI验证

Mooncake 上游 draft PR 状态如表6所示。

表 6 PR CI 状态

项目	内容
PR 链接	https://github.com/kvcache-ai/Mooncake/pull/2797
头提交	0123fa1 Fix fragmentation-aware allocation test setup
CI 结果	26个检查成功，1个检查跳过
结论	上游构建、格式检查、文档构建和相关测试已通过。

图5展示PR 合并检查区域：26项检查成功，1项按条件跳过，页面明确给出“All checks have passed”。跳过项是按触发条件不执行的自动检查，不表示构建或测试失败。

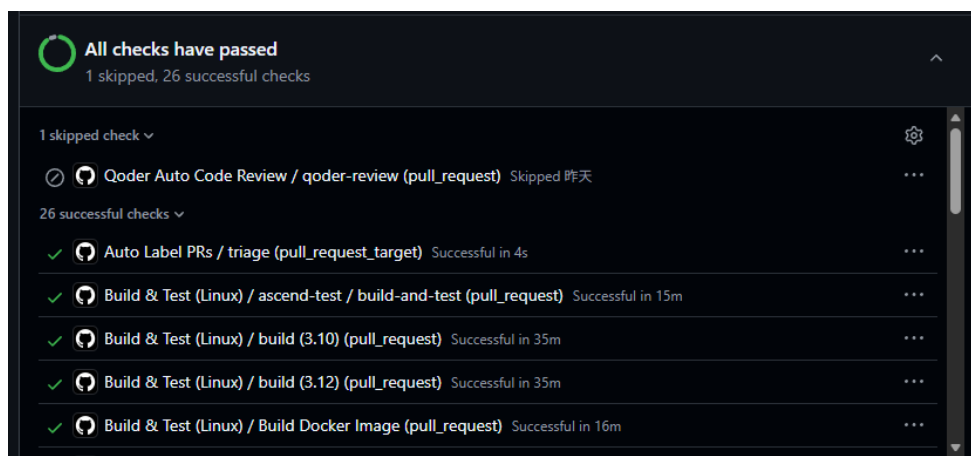


图 5 PR#2797 全部必需检查通过

图6进一步展示Linux workflow中的构建矩阵。Python 3.10/3.12 构建、ARM64 构建、不同编译标志、Docker 镜像、wheel 以及 CI Gate 等任务均显示为成功，说明改动没有局限在单一编译组合中。

图7是build (3.10) 任务中的测试汇总，页面显示90项测试全部通过、0项失败。该结果与定向碎片化测试共同构成“目标行为正确”和“既有功能未回归”两类证据。

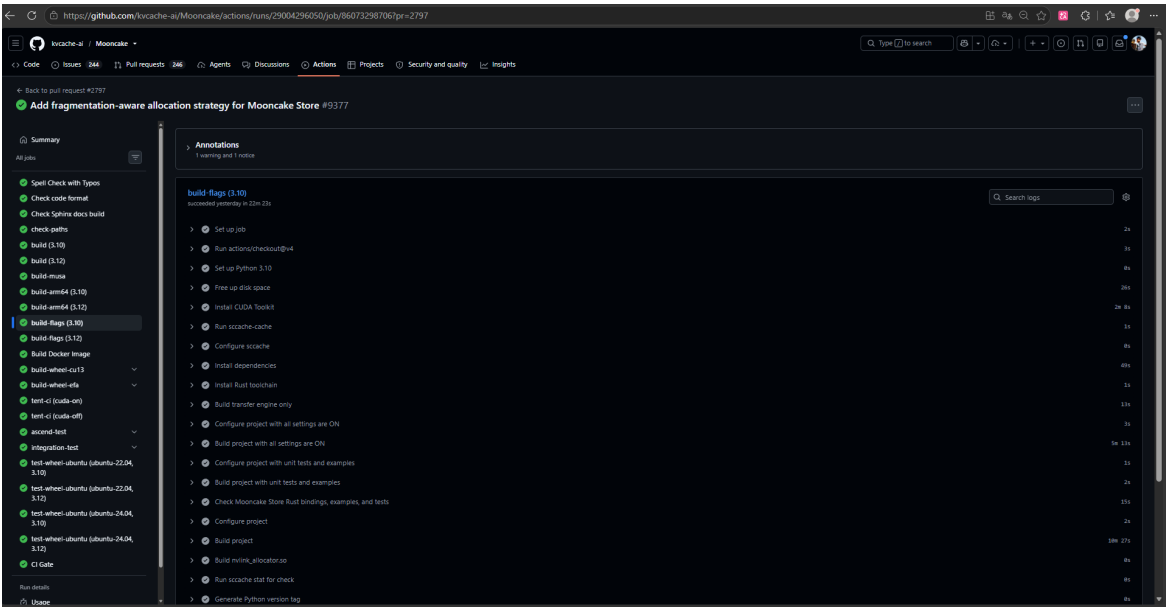


图 6 GitHub Actions Linux 构建与多配置任务成功

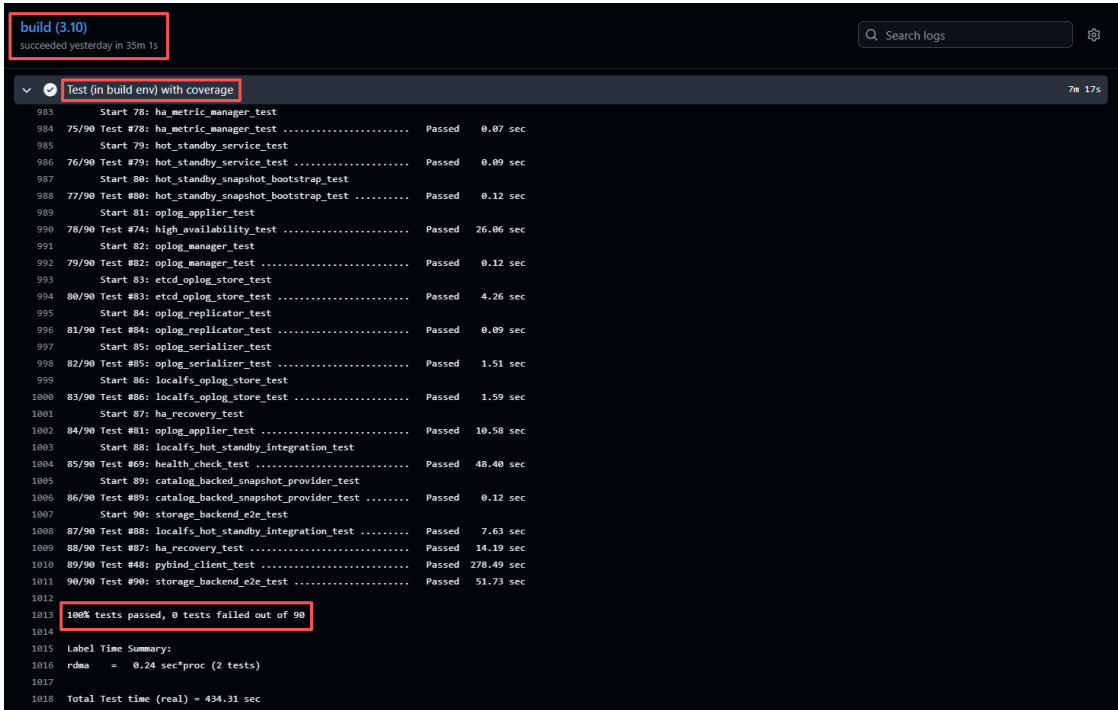


图 7 build (3.10)环境下90项回归测试全部通过

5.3 专题仿真指标

专题仿真使用 5 个 segment 和 6 个大对象请求，模拟高总空闲但碎片化、低总空闲但连续空间充足两类 segment 并存的 Store 状态。结果如表 7 所示。

这里的“首选 segment 可直接容纳”指排序后的第一个候选是否能够立即满足当前请求。这个指标非常关键，因为它衡量的是策略排序质量，而不是系统最终是否能够通过 fallback 补救。“最终可容纳”表示在继续尝试其他候选后，请求最终是否能找到空间；它用于确认两个策略面对同一容量条件时并没有改变总可用容量。“fallback 尝试次数”则用于衡量首选错误带来的额外路径压力。平均候选 segment 数量用于证明新策略没有通过扩大搜索范围获得收益，而是在相同候选规模内改进排序。

表 7 专题对齐仿真结果

指标	free_ratio_first	fragmentation_aware
大对象首选 segment 可直接容纳	0/6	6/6
最终可容纳	6/6	6/6
fallback 尝试次数	11	0
平均候选 segment 数量	5.00	5.00
单次决策耗时	121.00ns	138.93ns
新增决策开销	不适用	17.93ns

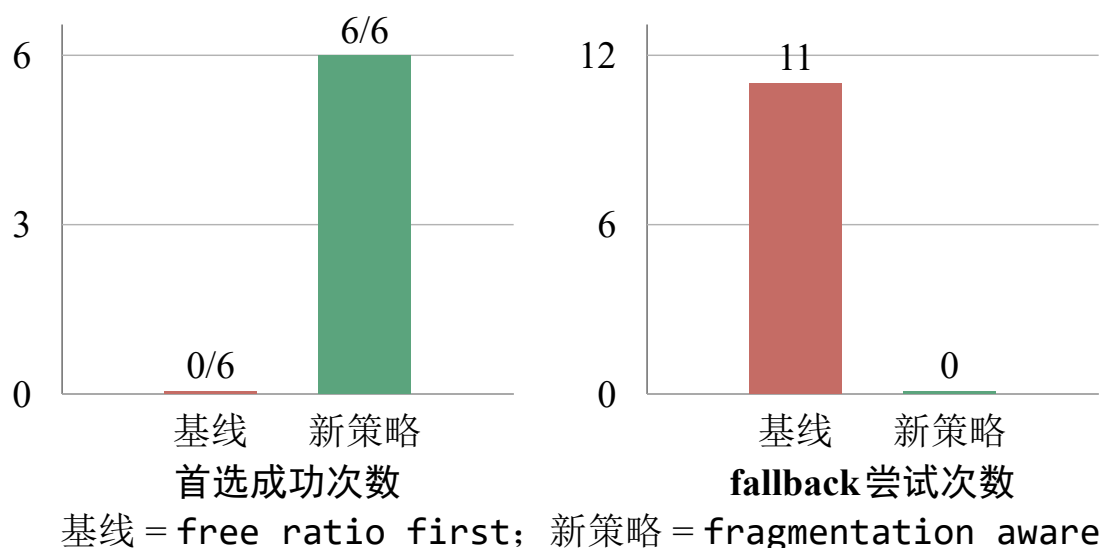


图 8 首选成功次数与 fallback 尝试次数对比

5.4 结果解释

该结果有三个含义。第一，`fragmentation_aware`显著改善首选segment质量，在所有测试请求上都能首先选中可容纳segment。第二，平均候选segment数量没有增加，说明改进来自排序信号变化，而不是扩大搜索范围。第三，单次决策新增开销约17.93ns，在本地CPU仿真中处于很小量级，符合“用少量元数据计算换取更稳定分配路径”的设计目标。

进一步看，`free_ratio_first`最终也能达到6/6可容纳，说明测试场景中系统总容量并不缺失；问题在于它先尝试了不合适的segment，需要额外fallback。换言之，本项目优化的不是“凭空增加容量”，而是“更快找到已经存在的正确位置”。这也是Store分配策略优化的现实价值：在容量足够但碎片化严重的状态下，吞吐和尾延迟问题往往来自无效尝试、重试和锁路径放大，而不一定来自物理资源绝对不足。

单次决策耗时从121.00ns增加到138.93ns，新增约17.93ns。这个成本主要来自读取最大连续空闲区域、计算连续性比例和增加排序比较字段。与一次真实网络I/O、远程存储访问或复杂推理请求相比，这一级别开销很小；但报告仍然保留该成本，而不把策略描述为“零开销”。更准确的结论是：`fragmentation_aware`用可控的纳秒级决策开销，换取碎片化场景下明显更高的首选成功率和更少fallback尝试。

5.5 证据有效性与威胁分析

从内部有效性看，专题仿真对两个策略使用相同的segment状态、请求序列、候选数量和计时方式，变化因素集中在候选排序信号。平均候选数量均为5.00，最终可容纳结果均为6/6，而首选成功和fallback次数出现明显差异，这说明收益不是来自增加物理容量、扩大搜索范围或降低请求难度，而主要来自`can_fit`与连续性信息改变了候选顺序。定向单元测试又使用另一条代码级证据验证了同一反例，降低了仅由仿真实现细节造成结论的风险。

从构念有效性看，“首选可直接容纳”和“fallback尝试次数”并不等同于端到端吞吐，但它们与本项目的直接优化目标一致：前者衡量排序后第一个候选的可行性，后者衡量错误首选带来的额外尝试路径。“最终可容纳”用于排除容量变化，“平均候选数量”用于排除搜索范围变化，“单次决策耗时”用于呈现策略自身成本。这组指标能够支持“分配尝试顺序更准确且额外计算受控”的结论，但不能单独推出真实集群P99延迟或每秒请求数提升比例。

外部有效性主要受负载和环境限制。当前确定性场景强调大对象与碎片化segment并存，尚未覆盖所有对象尺寸分布、淘汰节奏、副本因子和冷热变化；本地CPU计时也未包含生产环境中的锁竞争、NUMA、RDMA、远程元数据访问、GPU推理压力和SGLang HiCache前端行为。此外，不同allocator提供的最大连续空闲信息精度不同，并发请求还可能使评分快照在实际申请前发生变化。因此，现阶段结果适合证明算法方

向、实现正确性和收益机制，不应被外推为所有硬件与负载下的固定吞吐增益。

项目通过四种方式降低上述威胁。其一，策略采用显式配置开关并保留旧策略，便于在同一部署中进行 A/B 对照和快速回滚；其二，最终分配继续由 allocator 确认，避免元数据误差转化为空间安全问题；其三，将策略纳入上游单元测试、benchmark 和完整 CI，使后续测试可以直接复用相同实现，而不是重写离线原型；其四，完整保留仿真源码、输入口径和日志，使评审者能够复算当前指标。决赛阶段应进一步在真实 Store 压力测试中报告吞吐、P50/P99、fallback 比例、失败分配率、空间利用率和 CPU 开销，并在 SGLang HiCache 端到端环境中验证策略层改善能否转化为系统级收益。

6 与赛题要求的对应

本项目选择的是赛题 2 中的 Mooncake Store 性能优化方向。赛题同时关注吞吐性能、高可用功能、可扩展性以及 SGLang HiCache 与 Mooncake Store 的协同表现，因此作品不能只给出一个脱离主仓库的算法原型，而需要回答三个问题：优化是否发生在真实 Store 路径中，新增逻辑是否保留既有系统语义，以及结果是否具有可复现、可审查的证据。`fragmentation_aware` 位于 Master 侧 segment 分配决策路径，直接作用于 KVCache 对象进入 Store 后的放置过程，因而与赛题要求具有明确的代码和场景对应关系。

在性能方面，本项目没有把“减少一次错误候选尝试”直接等同于生产吞吐提升，而是先验证导致吞吐不稳定的具体中间机制。碎片化场景中，`free_ratio_first` 可能选择总空闲比例较高但无法容纳当前对象的 segment，随后发生 allocator 拒绝和 fallback。新策略通过 `can_fit` 和最大连续空闲区域提高首选候选的可行性，专题仿真中 fallback 尝试次数从 11 次降至 0 次。该结果证明分配路径中的无效工作可以被消除，为后续真实 Store 吞吐和尾延迟改善提供了可解释基础。

在可扩展性和高可用语义方面，本项目刻意保留 Mooncake 已有边界。策略只在现有有界采样集合内排序，平均候选数量保持 5.00，不随全局 segment 数量执行线性扫描；`preferred`、`excluded`、多副本去重和 `best-effort fallback` 语义均保持不变；最终空间申请继续由 allocator 完成。由此，新策略不会因为碎片元数据不精确而绕过空间安全检查，也不会把性能优化建立在扩大搜索范围或削弱副本约束之上。

对于 SGLang HiCache 场景，本项目优化的是其下游 Mooncake Store 后端的对象放置稳定性。HiCache 把需要保存或复用的 KVCache 对象交给 Store 后，首选 segment 是否可直接容纳对象，会影响后端写入路径是否进入额外尝试和 fallback。当前证据尚不能替代真实 HiCache 端到端压测，但已经完成了后端策略、配置、测试和 benchmark 入口，为后续使用真实推理负载测量缓存写入吞吐、P99 延迟和复用效果提供了可直接启用的实现基础。

表 8 赛题要求映射

赛题关注点	本项目实现	当前证据
Mooncake Store 吞吐性能	减少碎片化导致的无效首选分配和fallback压力	专题仿真中fallback尝试次数从11降为0
可扩展性	保持有界候选采样，不做全局扫描	平均候选segment数量保持5.00
高可用语义	不改变副本best-effort路径，保留preferred/excluded语义	单元测试和策略设计说明
SGLang HiCache+Store	优化Store后端对象放置路径，为HiCache后端稳定性提供基础	方案定位和代码路径位于Mooncake Store
开源规范	以PR形式提交上游Mooncake，并保留patch和文档	PR#2797通过CI
可复现性	提供仿真源码、日志、技术报告和使用说明	repro/、logs/、EVALUATION.md

6.1 评审证据链

赛题评分中的技术完整性不仅取决于代码数量，还取决于问题、实现和验证是否闭环。本项目的证据链可以按四层理解。第一层是问题证据：最小反例和专题模型表明总空闲空间与最大连续空闲区域并不等价。第二层是实现证据：PR在策略类型、配置解析、工厂创建、核心排序、测试、benchmark和文档路径中完成正式接入。第三层是正确性证据：定向单元测试覆盖碎片化选择与preferred语义，上游CI覆盖代码格式、构建、文档和回归测试。第四层是效果证据：确定性仿真在相同候选规模下比较首选成功、fallback次数和决策开销，并公开源码和日志供复算。

开源规范性则通过公开材料仓库、Mooncake fork分支和上游PR共同体现。KV-Cache Forge队伍在GitHub和GitLink均设置了公开比赛材料入口：GitHub仓库用于持续维护中文报告、PPT、复现程序、日志和提交压缩包，GitLink仓库用于赛事平台关联与队伍材料展示；Mooncake fork保留完整开发分支；上游PR#2797展示代码差异、提交历史和GitHub Actions结果。这些入口分别回答“成果如何阅读”“赛事材料从何访问”“代码如何复核”和“上游工程是否接受验证”等问题。相比只提交一个patch文件，该组织方式更便于评审者追踪来源、核对许可证并复现关键结论。

创新性主要体现在把碎片形态引入请求相关的候选排序，而不是简单新增一个静态容量阈值。can_fit把请求大小与最大连续空闲区域直接关联，综合score再在同一可容纳状态内兼顾连续性与空间均衡；同时采用显式开关、有界采样和allocator最终

确认，将算法收益与工程风险控制结合起来。该创新边界清楚：它不是全局内存整理器，也不修改Mooncake协议，而是对Store主路径中可独立评审、可低风险合入的决策环节进行针对性优化。

7 使用与复现

本项目提供两条互补的复现路径。第一条是Mooncake正式代码路径，适用于已经具备Mooncake Store构建环境的评审者：应用PR补丁、重新构建Master，并通过启动参数显式启用新策略。第二条是轻量专题复现路径，只依赖支持C++17的编译器，用于快速重现碎片误判、候选排序和指标统计。正式路径验证工程接入和真实接口兼容，轻量路径验证问题模型和算法机制，两者使用同一组核心概念，但承担的证据职责不同。

复现前应确认基础环境为主流Linux发行版，编译器支持C++17，并准备与PR基线兼容的Mooncake源码。若只运行`repro/`目录下的确定性程序，则不要求GPU、RDMA或完整SGLang环境。所有命令都应在仓库根目录执行，日志建议保存到独立文件，以便与`logs/`中的参考输出逐项比较。复现成功的最低判定标准是：程序正常退出、关键指标字段完整，并能观察到`fragmentation_aware`相对`free_ratio_first`的首选成功改善，同时候选数量保持不变。

7.1 启用策略

完成包含本项目改动的Mooncake Store构建后，可以在启动Master时通过命令行参数显式选择新策略：

```
mooncake_master --allocation_strategy=fragmentation_aware
```

该参数只改变Master侧候选segment的尝试顺序，不改变客户端协议、对象格式或已有数据。为了保证兼容性，Mooncake默认策略仍保持原值；只有明确传入`fragmentation_aware`时才启用新逻辑。实际灰度时应同时记录分配失败次数、fall-back次数、写入延迟和CPU利用率，并与相同负载下的`random`或`free_ratio_first`实例进行对照。

7.2 应用补丁

对于没有直接检出fork分支的评审者，可以在与PR基线兼容的Mooncake源码树中应用提交包提供的补丁。第一条命令只检查补丁上下文和文件路径，不修改工作区；检查通过后再执行第二条命令，能够避免在基线不匹配时产生部分应用状态。

```
git apply --check mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

```
git apply mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

应用后建议先检查 `git diff --stat` 和 `git diff --check`, 确认改动范围与报告中的 9 个文件一致且不存在空白字符错误; 随后按照 Mooncake 官方构建方式编译 Store, 并运行 `allocation_strategy_test`。如果补丁检查失败, 通常意味着本地 Mooncake 基线已经发生变化, 此时应优先检出 PR 对应提交或直接使用公开 fork 分支, 而不是强制应用补丁。

7.3 回滚补丁

若补丁尚未与其他本地修改混合提交, 可以使用反向应用恢复到应用前状态:

```
git apply -R mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

生产或灰度环境通常不需要回滚二进制, 只需将 `--allocation_strategy` 切换回原有策略并重启对应 Master 实例, 即可恢复旧的候选选择行为。补丁级回滚主要用于源码评审和独立实验清理。执行反向应用前仍建议先运行 `git apply -R --check`, 避免覆盖评审者在相同文件中的其他修改。

7.4 复现实验

轻量实验由三个程序组成。`fragmentation_aware_sim.cpp` 用于演示最小碎片化反例; `fragmentation_aware_metrics.cpp` 统计首选成功、`fallback` 和决策耗时; `topic_aligned_store_scalability_sim.cpp` 使用赛题对齐场景验证在固定候选规模下的策略行为。使用 `-O2` 是为了减少调试构建带来的计时噪声, 所有程序均使用固定输入, 便于不同机器重复执行。

```
g++ -std=c++17 -O2 repro/fragmentation_aware_sim.cpp -o
    fragmentation_aware_sim
./fragmentation_aware_sim

g++ -std=c++17 -O2 repro/fragmentation_aware_metrics.cpp -o
    fragmentation_aware_metrics
./fragmentation_aware_metrics

g++ -std=c++17 -O2 repro/topic_aligned_store_scalability_sim.cpp -o
    topic_aligned_store_scalability_sim
./topic_aligned_store_scalability_sim
```

执行后应重点核对四类输出。第一, `free_ratio_first` 的大对象首选可容纳次数为 0/6, 而 `fragmentation_aware` 为 6/6; 第二, 两者最终可容纳次数均为 6/6, 证明

物理容量条件没有改变；第三，`fallback` 尝试次数由 11 降为 0；第四，平均候选数量均为 5.00，证明收益并非来自扩大搜索范围。决策耗时可能随 CPU 型号、频率和系统负载变化，因此 17.93ns 应作为当前参考环境下的观测值，而不是跨硬件固定常数。若功能指标一致而绝对耗时略有波动，仍可认为问题机制和策略效果得到复现。

8 边界与后续工作

本项目当前阶段已经完成代码实现、上游 PR、CI 验证、仿真评估和中文提交材料整理，但仍有边界需要明确：

- 当前不宣称已经完成真实 RDMA 硬件 benchmark。
- 当前不宣称已经完成真实 SGLang HiCache 端到端吞吐压测。
- 当前不宣称已经重设计 Mooncake 高可用协议。
- 当前专题仿真验证的是分配策略层，不等同于完整生产集群压测。

后续如果进入决赛，建议补充三类工作：第一，在真实 Mooncake Store 环境中运行 `allocation_strategy_bench`，统计吞吐、P50/P99、失败分配次数、`fallback` 次数和内存利用率；第二，在具备 RDMA 条件的集群中验证跨节点场景；第三，将 SGLang HiCache 接入 Mooncake Store 后端，使用真实推理负载验证 KVCache 复用和后端 IO 表现。

9 结论

本文围绕 Mooncake 赛题 2 提出并实现 Mooncake Store 碎片感知分配策略。该策略针对 `free_ratio_first` 只看总空闲比例的不足，引入最大连续空闲区域和可容纳性排序，在不改变默认行为、不扩大候选规模、不破坏原有副本语义的前提下，提升碎片化场景下的大对象首选分配质量。当前 PR 已经通过上游 GitHub Actions，确定性仿真显示首选成功次数从 0/6 提升到 6/6，`fallback` 尝试次数从 11 降到 0。该工作是一个边界清晰、可复现、可回滚、适合继续扩展到真实 Store benchmark 和 SGLang HiCache 端到端测试的初赛成果。

10 提交材料清单

文件或目录	说明
README.md	项目入口、快速使用和成果索引
SUBMISSION.md	平台提交说明与必交材料对应关系
DESIGN.md	问题、目标、算法与兼容性设计
EVALUATION.md	CI、仿真指标和结论边界
demo_script.md	5分钟作品展示逐字稿与分镜
mooncake_fragmentation_aware pr_2797_0123fa1.patch	当前PR补丁
repro/	可独立编译的复现实验源码
logs/	三份关键仿真结果日志
tools run_powershell_demo.ps1	Windows PowerShell 一键演示脚本
slides/	初赛展示PPT 最终版
report/	本技术报告源码、PDF和证据截图
release/	提交压缩包和校验文件

11 参考链接

- Mooncake 上游仓库: [kvcache-ai/Mooncake](#)
- 本项目 Mooncake PR: [kvcache-ai/Mooncake#2797](#)
- GitHub 比赛材料仓库: [Lorry1024/ccf2026-mooncake-fragmentation-aware](#)
- GitLink 比赛材料仓库: [lorry22/ccf2026-mooncake-fragmentation-aware](#)
- Mooncake fork 分支: [Lorry1024/Mooncake:ccf-fragmentation-aware-allocation](#)
- 作品展示视频: 百度网盘, 提取码 mxhx